



Bridging natural language and GIS: a multi-agent framework for LLM-driven autonomous geospatial analysis

Ali Mansourian & Rachid Oucheikh

To cite this article: Ali Mansourian & Rachid Oucheikh (2026) Bridging natural language and GIS: a multi-agent framework for LLM-driven autonomous geospatial analysis, International Journal of Digital Earth, 19:1, 2633849, DOI: [10.1080/17538947.2026.2633849](https://doi.org/10.1080/17538947.2026.2633849)

To link to this article: <https://doi.org/10.1080/17538947.2026.2633849>



© 2026 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.



Published online: 23 Feb 2026.



Submit your article to this journal [↗](#)



Article views: 2546



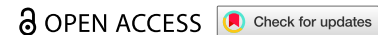
View related articles [↗](#)



View Crossmark data [↗](#)



RESEARCH ARTICLE



Bridging natural language and GIS: a multi-agent framework for LLM-driven autonomous geospatial analysis

Ali Mansourian and Rachid Oucheikh

Earth and Environmental Sciences, Lund University, Lund, Sweden

ABSTRACT

Existing LLM-based approaches remain limited by simplistic task execution, restricted tool integration, and a lack of contextual reasoning when interacting with professional GIS software. This study investigates the efficacy of a multi-agent architecture designed to enhance geospatial task execution accuracy through collaboration, reasoning and tool-use. The architecture integrates Chain of Thought (CoT) reasoning and Retrieval-Augmented Generation (RAG) and employs specialized agents that collaboratively translate high-level natural language queries into structured, executable workflows using QGIS processing algorithms as tools. Through a structured fine-tuning approach, we evaluated how the multi-agent setup influences spatial task comprehension, geoprocessing tool selection, and code generation. The results demonstrate that the developed framework significantly outperforms baseline single-agent and non-fine-tuned systems. For tasks involving one or two GIS tools, the system achieved up to 100% execution success and 87.5% semantic correctness. However, its effectiveness decreases with more complex, multi-step workflows. Notably, iterative self-refinement and self-debugging led to moderate gains in execution success and semantic correctness. The results indicate that multi-agent frameworks are a promising approach, even though improvements in reasoning depth and tool alignment are still needed. This study represents a foundational step toward more reliable, modular, and adaptable LLM-based systems for geospatial analysis automation.

ARTICLE HISTORY

Received 20 June 2025
Accepted 14 February 2026

KEYWORDS

Large Languages Models (LLMs); AI Agent; autonomous geospatial analysis; GIS; GeoAI

1. Introduction

Geospatial analysis is essential across many domains but remains complex and inaccessible to non-experts, a gap that emerging Large Language Model (LLM)-based solutions can bridge. LLMs have emerged as transformative tools in natural language processing (NLP), demonstrating remarkable capabilities in creativity, linguistic proficiency, and problem-solving across diverse domains (Hadi et al. 2023). Being pre-trained on vast text and code datasets, LLMs excel not only on the mentioned text-based tasks, but also with code generation and problem solving (Raiaan et al. 2024). This ability to interpret complex tasks and generate functional solutions positions them as promising intermediaries between human users and GIS tools and potential core for solutions enabling intuitive interaction with geospatial data and workflows. Realising this potential, however, necessitates a clear understanding of their inherent geospatial capabilities, which led recent studies to focus on evaluating their embedded geographic knowledge and GIS functionality. For instance, Xu et al. (2025b) show that although large language models demonstrate strong performance in foundational geographic and GIS-concept tasks (with accuracy above 80%) they still struggle substantially on more complex spatial and operational tasks (such as route-planning and grid-based spatial reasoning), where performance drops sharply as task complexity increases. Furthermore, while LLMs demonstrate competence in qualitative spatial reasoning (e.g. ordinal relationships), quantitative (metric) and complex topological reasoning remain challenging (Wang et al. 2024). These findings collectively indicate that, although LLMs possess some latent geospatial knowledge, substantial advancements in spatial reasoning, accuracy, and external

CONTACT Ali Mansourian ali.mansourian@mgeo.lu.se Earth and Environmental Sciences, Lund University, Sölvegatan 12, SE-223 62, Lund, Sweden

© 2026 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.

This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial License (<http://creativecommons.org/licenses/by-nc/4.0/>), which permits unrestricted non-commercial use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

data integration are necessary before they can be considered reliable alternatives or even robust assistants for complex GIS applications.

Recent studies have focused on evaluating and extending LLM capabilities for geospatial tasks. Mai et al. (2023) applied LLMs to toponym recognition, location description extraction, and US dementia time series forecasting using task-specific text inputs. Hu et al. (2023) introduced a geo-knowledge-guided GPT model to extract detailed location descriptions from social media using prompt-based training examples. A major development is the creation of frameworks for adapting LLMs to geospatial tasks and autonomous GIS agents. Mansourian and Oucheikh (2024) developed ChatGeoAI, a fine-tuned LLaMA 2 model that translates natural language queries into executable PyQGIS code. Despite challenges with ambiguous attribute names, the system shows promise in handling geometries, spatial relationships, and attribute queries. Hou et al. (2025a) proposed the Chain-of-Programming (CoP) framework to translate user requests into Google Earth Engine (GEE) scripts. By breaking code generation into steps like requirement analysis, algorithm design, and debugging, CoP improves logical structure, syntax, and executability.

Several studies have explored extending LLM applications by leveraging their ability to use external tools. Rather than relying solely on inherent knowledge for direct output generation results, LLMs can interpret tasks and plan the strategic use of external resources. Wei et al. (2024) designed GeoTool-GPT to allow LLMs to master GIS tools. The LLM is trained on a structured dataset of GIS-specific instructions and tool-use workflows (GeoTool, GeoSolution, and GeoTask) and showed improvement in its ability to understand GIS tools, generate correct tool sequences, and solve geospatial analysis tasks. Although GeoCode-GPT (Hou et al. 2025c) focuses on generating high-quality geospatial code, it lacks direct execution capabilities and requires users to manually run, debug, and refine scripts. In addition, it does not inherently provide contextual decision-making or adaptive error handling. Moreover, since the solution is optimised for geospatial code generation across multiple platforms, it does not offer an interactive autonomous workflow execution. A related system is GeoGPT which employs an LLM as an autonomous agent to interpret geospatial queries and execute predefined GIS tools (Zhang et al. 2024b). Built using the LangChain framework, it connects LLMs (e.g. GPT-4) with a GIS tool pool containing functions for data collection, spatial analysis, and mapping. While GeoGPT demonstrates the potential of LLMs for automating geospatial workflows, it has several limitations that our system aims to address. Firstly, GeoGPT relies on predefined GIS tools which limit its flexibility when handling new or complex tasks that require dynamic code generation. Secondly, it lacks fine-tuned domain-specific training which makes it more prone to execution errors and requires careful tool integration.

Despite these advancements, current LLM-driven geospatial systems exhibit critical limitations that hinder their practical adoption in professional GIS workflows that can be summarised as:

- (1) Existing approaches predominantly rely on rigid, predefined tool libraries, which lack adaptability to novel tasks requiring dynamic code synthesis or contextual tool orchestration.
- (2) The absence of integrated reasoning mechanisms, such as systematic decomposition of multi-step geospatial queries or real-time validation of spatial logic, often results in error-prone outputs.
- (3) While LLMs exhibit nascent geospatial knowledge, their limited integration with domain-specific data (e.g. GIS tool documentation, geospatial ontologies) restricts their ability to generate accurate, context-aware workflows.
- (4) Current frameworks treat geospatial task execution as a single-step code generation problem, overlooking the iterative, feedback-driven nature of real-world spatial analysis where intermediate results often necessitate parameter adjustments or algorithmic refinements.

These gaps highlight the need to investigate how a multi-agent approach can effectively integrate LLMs' generative abilities with structured spatial reasoning, domain grounding, and autonomous execution. Therefore, our primary aim is to explore how multi-agent systems can improve geospatial tool-use in open-source LLMs.

Geospatial tasks often involve interconnected tools executed in a specific order, where outputs from one tool become inputs for subsequent ones. This sequential dependency introduces complexities in reasoning, workflow building, fine-tuning, and debugging processes. To address these challenges, we propose investigating a multi-agent framework, with each agent specialised in particular functions and designed

to interact seamlessly, facilitating efficient collaborative execution of complex geospatial workflows. Specifically, our contributions include:

- Multi-agent framework for geospatial reasoning: Development and assessment of novel architecture consisting of specialised agents responsible for task decomposition, contextual tool retrieval, code generation, and iterative debugging. This framework aims to enhance collaborative problem-solving capabilities in spatial workflows.
- Domain-adapted LLM training: An LLM fine-tuning methodology which combines reasoning-focused training on geospatial tasks with instruction tuning for executable PyQGIS code generation. This methodology seeks to investigate improvements in both spatial logic comprehension and accuracy in tool-specific implementations, thereby aiming to minimise code hallucination.
- Autonomous execution pipeline: Implementation and analysis of an integrated workflow leveraging CoT reasoning combined with RAG to dynamically convert natural language queries into validated PyQGIS scripts. The pipeline's effectiveness will be evaluated through real-time error correction facilitated by continuous feedback loops among specialised agents.
- Specialised geospatial dataset: A benchmark corpus of geospatial tasks, each annotated with step-by-step reasoning chains, tool dependencies, and executable code, which can facilitate reproducible training and evaluation of LLMs in GIS automation.

To make these contributions, the following research questions should be answered:

- (1) How effectively can a multi-agent framework synergise structured reasoning (CoT) and domain knowledge retrieval (RAG) to decompose geospatial queries into correct workflow steps and executable code?
- (2) To what extent does fine-tuning specialised multi-agents on domain-specific data improve the accuracy and reliability of the generated PyQGIS code compared to baseline LLMs?
- (3) What are the practical limitations of LLM-driven geospatial automation when handling tasks with different levels of complexity and involving various numbers of GIS tools?

By systematically investigating these questions, our work advances the development of autonomous GIS assistants capable of bridging natural language interaction with professional-grade spatial analysis, thereby democratising access to advanced geospatial methodologies.

The subsequent sections of this paper are organised as follows. [Section 2](#) provides background information on the use of LLMs for geospatial analysis and code generation, also discussing relevant aspects of the used approaches. [Section 3](#) describes the materials and methods employed, including the geospatial analysis problem context, system proposed, model selection, and evaluation metrics. Results are presented in [Section 4](#), followed by a detailed discussion in [Section 5](#). [Section 6](#) outlines the study's limitations and opportunities for future research. Finally, [Section 7](#) concludes the paper with final remarks and recommendations.

2. Background

2.1. Large language models in GIS

Recent advances in large language models have expanded their role beyond text generation toward domain-adapted reasoning, tool integration, API use, and complex task execution. In the GIS domain, this evolution has driven increasing interest in embedding LLMs within GIS workflows to support automation, spatial reasoning, and interaction with specialised geospatial tools. Consequently, recent research has explored a range of LLM-enabled GIS capabilities, including GIS tool interaction and automation (Wei et al. 2024; Akinboyewa et al. 2025), enhancing geospatial reasoning (Wang et al. 2024; Tucker 2024), extracting spatial data and building knowledge graphs, understanding and generating spatial language (Jiang and Yang 2024; Mansourian and Oucheikh 2024), and developing evaluation benchmarks (Bhandari, Anastasopoulos, and Pfoser 2023; Xu et al. 2025b). Although LLMs can encode

some geospatial knowledge, their capacity for synthesis and reasoning remains constrained. Early studies assessed spatial literacy using designed prompts and evaluation metrics to test understanding of direction, distance, adjacency, and containment (Manvi et al. 2024).

Some studies have tried to leverage LLMs for their semantic reasoning, domain knowledge, and spatial analysis capabilities. Mai et al. (2023) applied LLMs to tasks such as toponym recognition and location description extraction, while Hu et al. (2023) developed a geo-knowledge-guided GPT for extracting spatial information from social media. Bhandari, Anastasopoulos, and Pfoser (2023) evaluated LLMs' spatial reasoning capabilities, such as understanding distance, containment, and adjacency, and found that larger models exhibit stronger spatial awareness, suggesting that LLMs can internalise basic geographic logic. To enhance such reasoning, Tucker (2024) reviewed geospatial embedding techniques, including Entity, Document, Sequence, and Token Location Embeddings, which improve spatial understanding. Despite these advances, current systems still face challenges in robust spatial reasoning and complex analysis, which motivates further research toward integrating LLMs into autonomous GIS frameworks.

Building on this motivation, another emerging line of research focuses on harnessing LLMs' reasoning abilities for automated geospatial code generation, which is a key step toward achieving autonomy in GIS workflows. LLMs have transformed automated code generation by translating natural language into executable programs, thereby enabling tasks such as code completion, debugging, and language translation (Jiang and Yang 2024). Benchmarks like HumanEval and SWE-Bench show top models achieving over 80% pass@1 accuracy, though performance drops on more complex tests (Liu et al. 2023). Despite this progress, LLMs often produce functionally incorrect or logically flawed code, particularly in domains requiring specialised reasoning such as GIS (Christakis and Drikakis 2025). Their limited exposure to spatial data and GIS libraries leads to hallucinated functions and API misuse. Recent work, such as GeoCode-GPT (Hou et al. 2024b), demonstrates that domain-specific fine-tuning markedly improves geospatial coding accuracy and paves the way for more reliable autonomous GIS systems.

Recent systems such as GeoCogent (Hou et al. 2025b) and GeoCoLab (Wu et al. 2025) have advanced LLM-driven GIS assistance by more explicitly structuring geospatial code generation workflows. GeoCogent extends domain-specific fine-tuning and retrieval-augmented generation by organising code synthesis into staged reasoning processes, including requirement interpretation, algorithm design, and code generation, which improves syntactic correctness and logical coherence across multiple geospatial platforms. However, its scope remains centred on producing code artifacts: while reasoning is staged for code synthesis, geospatial workflows are not executed or validated within the system, and correctness ultimately depends on manual execution and user-driven debugging. GeoCoLab, in contrast, emphasises collaborative and agent-based assistance for composing and reasoning about geospatial workflows, supporting users through interactive role-based agents in a human-in-the-loop setting. While this design facilitates exploratory analysis and collaborative workflow construction, it similarly does not support autonomous execution, data-aware validation, or systematic error recovery. Consequently, both approaches address important aspects of LLM-based GIS assistance but stop short of full operational autonomy, as they lack integrated execution-aware feedback, workspace-level data introspection, and self-healing mechanisms. These limitations directly motivate our work, which targets the next step beyond code generation by enabling end-to-end autonomous geospatial analysis through multi-agent reasoning, executable workflows, and iterative self-correction.

2.2. Autonomous GIS using LLMs

Li and Ning (2023) introduced the concept of Autonomous GIS, where LLMs serve as the core reasoning engine, autonomously generating and executing commands or code for tasks such as spatial data collection, analysis, and visualisation. Their prototype, LLM-Geo, effectively produces outputs like aggregated data, graphs, and maps. A fully autonomous GIS system should be self-generating, self-organising, self-verifying, self-executing, and self-improving. In practice, agents have been developed with varying autonomy across different stages of the geospatial workflow. As example, for autonomous data retrieval, Ning et al. (2025) proposed LLM-Find, an agent framework where an LLM selects and retrieves data from a predefined list of sources. The prototype demonstrated a success rate of 80–90%.

Recent advancements propose integrating LLMs with external tools to overcome domain-specific limitations. Instead of relying solely on internal knowledge, LLMs can function as intelligent agents that interpret tasks and invoke specialised tools as needed (Yao et al. 2023; Wei et al. 2024). By accessing GIS libraries, databases, or APIs, LLMs can handle precise operations (e.g. calculations or syntax) while using their language capabilities to determine tool usage. For instance, GeoAgent connects an LLM to geocoding APIs for address parsing, improving accuracy on free-form inputs (Huang et al. 2024). Similarly, MapGPT automates map-making by decomposing maps into elements, each handled by a dedicated tool (Zhang et al. 2024a). The LLM orchestrates these tools based on natural language input. For more specific tasks, Lin et al. (2024) proposed ShapefileGPT, a multi-agent framework that distributes shapefile processing across LLM-powered agents and showcase collaborative task execution in GIS. While promising, this approach requires further research into agent coordination. In the same context, GeoLLM-Engine (Singh, Fore, and Stamoulis 2024) provides a realistic development and evaluation environment for building geospatial copilots with the aim to offer standardised benchmarks, reproducible workflows, and a safe interface for tool interaction. Such platforms support the design of autonomous GIS systems by allowing controlled experimentation with agent coordination and reasoning strategies.

Autonomous approaches commonly employ prompt engineering, knowledge bases, and chain-of-thought techniques to enable LLMs to perform tasks and workflows. However, full autonomy remains challenging due to issues with data handling, inconsistent outputs, computational overhead, and lack of transparency, which can reduce user trust. LLMs also require ongoing oversight due to variable performance. While these models can generate syntactically correct code, they often produce semantic errors, e.g. failing to perform intended GIS operations or misusing APIs. Additionally, many implementations rely on proprietary models like GPT-4, which limit transparency and reproducibility (Wang et al. 2024). Although open-source LLMs perform well in natural language tasks, they lag in tool-use capabilities (Xu et al. 2025a). Recent efforts, such as Geo-OLM (Stamoulis and Marculescu 2025), explore state-driven reasoning that divides workflows into discrete operational states to improve efficiency and reduce computational overhead in open models. However, such approaches remain experimental and highlight the broader challenge of balancing cost-efficiency with reliable tool integration. High computational demands and reliance on human oversight also hinder scalability (Zhang et al. 2025). These challenges highlight the need for adaptable frameworks that align LLMs with geospatial tools while emphasising accessibility and reliability.

In summary, key limitations hinder the widespread adoption of LLMs in GIS, including weak geospatial knowledge, inconsistent performance, limited spatial reasoning, and difficulties with geospatial formats and coordinate systems. So, overcoming these challenges may require a collaborative system of specialised agents, each focusing on a distinct competence such as spatial reasoning, knowledge augmentation, semantic interpretation, or error correction. Accordingly, this paper introduces a multi-agent framework designed to decompose complex geospatial queries into manageable subtasks, align appropriate tools, and iteratively refine execution outcomes. Unlike existing frameworks (e.g. GeoCode-GPT, GeoTool-GPT, and CoP), our approach emphasises modular agent collaboration, retrieval-grounded decision-making, and self-debugging cycles with the aim of improving reliability and workflow automation in GIS.

3. Method and materials

3.1. Data collection

A key approach is the development of specialised training datasets. The idea is that general pre-trained LLMs often lack specific geospatial knowledge and may not perform well in the geospatial domain. Therefore, it is required to create specific datasets that can be used to fine-tune the models. These datasets may contain geospatial text data, maps, geographic relationships, and examples of geospatial analysis tasks. The content of these datasets depends on the target task and the goal of fine-tuning. This involves careful selection of the data and prompt engineering which are often a time-consuming and expensive task. The first collected data includes 500 samples and geospatial tasks related to data extraction and processing, projection and coordinate systems, vector analysis and geoprocessing, raster processing, data cleaning and transformation, change detection, and other miscellaneous GIS operations. The dataset was balanced to

include an equal number of queries across four levels of complexity, defined by the number of tools involved (1, 2, 3, 4, 5, 6, and 7), to ensure fair evaluation across varying workflow lengths. The classification of geospatial analysis task difficulty is not strictly ordinal, since the same analytical goal can often be achieved through different valid tool sequences. For instance, a task solved in four tools can be solved using five or even more tools and the generated workflows represent comparable complexity. For lower number of tools, the class separability is stricter. For this reason, we grouped all workflows involving four to seven tools into a single high-complexity category. But this is not to say that an approximate ranking of GIS workflows is irrational; it still serves as a practical proxy for assessing LLM performance, since higher-complexity tasks generally demand stronger reasoning, longer dependency chains, and more precise parameter alignment. To obtain a fair assessment, we therefore used multiple complementary evaluation metrics and manually inspected all outputs to verify whether both the generated results and the selected tools were appropriate.

The dataset is collected manually using mostly OSM-derived layers, publicly shared layers, and simple manually generated layers. Most of the codes were created manually with the official QGIS Processing Algorithms documentation was used as a reference to ensure broad coverage of essential geoprocessing operations. All 500 Query–CoT–Code triplets were executed in the QGIS Python console to verify syntactic and semantic validity. Only workflows that produced correct and complete outputs were retained to ensure the dataset consists exclusively of executable and validated tasks. Then, the dataset is split to fine-tuning and test set while the primary objective was to maintain the test set’s representativeness relative to the dataset used for fine-tuning. Each sample contains a prompt, Chain-of-Thought (CoT) steps, and code:

- Prompt: A geospatial task description in natural language with objective and constraints, typically involving GIS operations like data extraction, transformation, and analysis.
- CoT (Chain of Thought): A step-by-step breakdown of the logical reasoning process to solve the task described in the prompt.
- Code: Python scripts written for QGIS to execute the described geospatial analysis or data processing task.

For example, [Table 1](#) presents a representative query: ‘Show rivers that cross through urban areas, and calculate the total length of rivers in this area.’ To solve this task, three main steps should be followed. First, the necessary data layers, namely rivers and urban areas, must be loaded. Next, two GIS processing operations will be performed: *intersection* to identify river segments within urban areas, *field calculator* to compute their lengths, and *basic statistics for fields* to compute the sum of the lengths. Finally, the results will be saved.

Chain-of-Thought (CoT) prompting is central to our approach. This technique instructs the LLM to generate intermediate reasoning steps before the final answer, aiding multi-step logical tasks common in geospatial workflows. CoT not only boosts performance but also improves interpretability, supports human verification, and enables structured input for multi-stage pipelines. Prior work (e.g. Wei et al. 2022) highlights its effectiveness, making it a key element of our training strategy.

3.2. System architecture

[Figure 1](#) depicts the proposed system is designed as an autonomous agent-based framework that translates natural language geospatial queries into executable PyQGIS scripts, iteratively refines the generated code,

Table 1. Collected data including query, task steps and code.

Query:	Task steps (CoT)
Show rivers that cross through urban areas, and calculate the total length of rivers in this area	Step 1. Data loading Step 2. Use the Intersection algorithm (native:intersection) to find river segments inside the urban areas. Step 3. Use the Field Calculator algorithm (native:fieldcalculator) to calculate the length of the identified river segments. Step 4. Use basic statistics (qgis:basicstatisticsforfields) to compute the sum of their lengths. Step 5. Save the results.

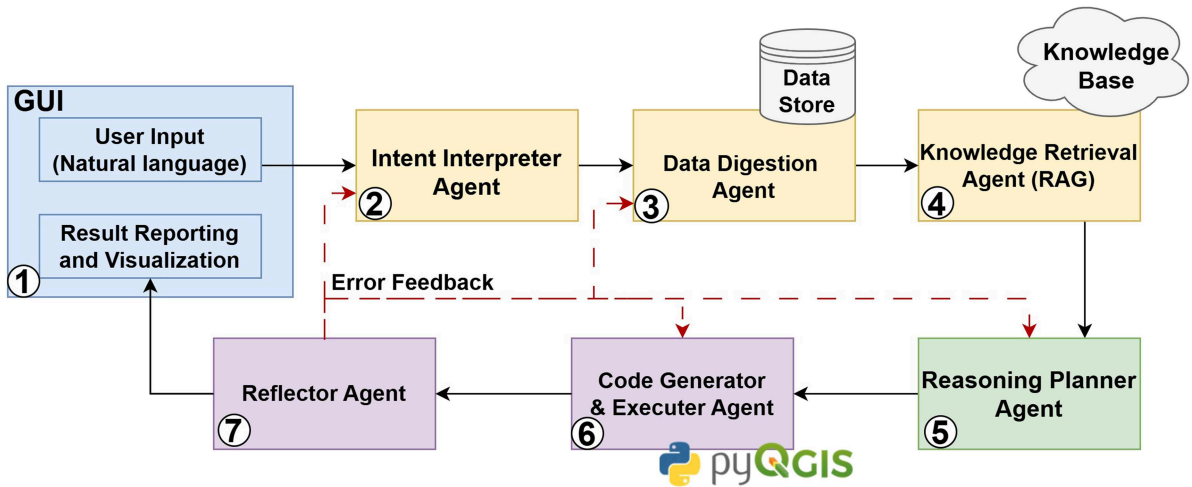


Figure 1. Architecture of the proposed system.

and executes it within a controlled environment. The architecture integrates RAG, multi-agent coordination, and an iterative code improvement loop, ensuring robustness and correctness in geospatial analysis workflows. The system consists of several interacting components, each fulfilling a specialised role while communicating seamlessly to generate, debug, and execute geospatial processing tasks. The pipeline proceeds sequentially from the Intent Interpreter (1) to the Data Digestion Agent (2), the Knowledge Retrieval Agent (3), and the Reasoning Planner (4). Outputs then flow to the Code Generator and Executor (5). If execution fails, the Reflector (6) sends targeted feedback to the relevant upstream agent, as indicated by dashed feedback arrows in the figure to start a self-correcting cycle.

At the core of the architecture lies a multi-agent system orchestrated to manage the entire query-to-execution pipeline. The process begins by providing a query in the user-friendly web interface (1). Then, the query is sent to Intent Interpreter agent (2), a LLaMA-3 model that produces a normalised schema that lists the task type, target entities, spatial predicates, grouping keys, and implicit constraints (e.g. CRS compatibility). Then, both the original query and the intent generated by the previous agent are sent to a Data digestion agent (3) that crawls the user workspace, inspects metadata (geometry type, CRS, schema), and returns a canonical list of candidate layers or flags missing data early. Then, a RAG agent (4) searches over a vector store containing PyQGIS documentation and curated execution traces supply authoritative passages that will be injected into the planner prompt. This agent leverages RAG by interacting with RAG framework (3) to enhance accuracy by retrieving the context through identification of relevant chunks of the documentation. This query undergoes context enrichment, where the system adds relevant information about available tools and data sources to guide the agent's decision-making process. Parameter extraction is performed implicitly by the language model, which identifies key elements such as target features, distances, and analytical operations from the natural language input. Afterwards, a Reasoning Planning agent (5) breaks down the query and the generated intent into a structured sequence of tasks using a Chain-of-Thought reasoning approach. It generates reasoning justification and a JSON plan that specifies each processing operation and its parameters. Subsequently, all the outputs from the previous agents and mainly the produced plan are fed to the Code Generator and Executor agent (6) which identifies the processing algorithms required to fulfil the plan steps, generates the code and runs it in the QGIS environment. Its output consists of the generated Python code and JSON file showing the execution logs and success status. Finally, the Reflector agent (7) receives the outputs of all the agents and tries to analyse the performance of the multi-agent geospatial pipeline by reflecting on logs and final outputs to diagnose the error source, identify issues and suggest improvements to the specific agent which caused the error. It returns a structured Json which includes a success flag, the responsible agent's ID, and a prompt specifying the corrective action that should be made by the responsible agent. All the agents use LLaMA-3-70B-Instruct as underlying LLM except the Planner and Code Generator agents which use DeepSeek-R1-Distil-Llama-70B for reasoning capabilities.

Table 2. Structured inputs and outputs exchanged among specialised agents. Each agent communicates through standardised JSON objects to flow traceability and interaction.

Agent	Input	Output
Intent Interpreter Agent	User query in natural language	Normalised JSON intent schema describing task type, spatial predicates, feature classes, and constraints
Data Digestion Agent	User query, JSON intent schema, user data directory	Metadata summary of the selected layers (paths, CRS, geometry types, attributes) with validation report in JSON
Knowledge Retrieval Agent	User query, JSON intent schema, metadata summary	Top-k retrieved text chunks from PyQGIS API docs & QGIS manuals, encompassed in JSON format
Reasoning Planner Agent	User query, JSON intent schema, metadata summary, retrieved knowledge	(i) Human-readable reasoning trace; (ii) Workflow JSON plan listing algorithms, inputs, parameters
Code Generator and Executor Agent	User query, JSON intent schema, metadata summary, retrieved knowledge, Workflow plan JSON	Execution-result JSON containing success/failure flag, logs, and produced layers
Reflector Agent	Execution-result JSON + previous agent outputs	Diagnostic JSON including success flag, faulty agent ID, and corrective prompt

The interaction among agents is fully message driven. Each component receives a structured JSON object as input, performs its designated operation, and returns a new JSON output that becomes the input for the next stage. This design provides a transparent communication protocol to enable deterministic orchestration and seamless fault tracing across the workflow. Table 2 summarises the standardised input–output specifications for all agents in the framework. A detailed breakdown of the functionality of each system component is provided in the following:

- (1) Graphical User Interface: The system’s web-based GUI serves as both the entry point for natural-language queries and the main channel for delivering results. Upon query submission, the GUI serialises the input and forwards it to the orchestrator, while maintaining a WebSocket connection for real-time feedback, including logs, warnings, data layers, and charts. The Visualizer, a stateless microservice, equipped with Folium, renders geospatial results into web-ready formats so to enable headless execution without sacrificing visual quality or interactivity. For example, a user can enter this query: *‘Identify rivers intersecting urban areas and calculate their total length.’*
- (2) Intent Interpreter Agent: Natural language geospatial queries exhibit multiple sources of ambiguity: polysemous verbs (e.g. *intersect*, *overlay*), implicit spatial relationships (e.g. *near*, *within*), and the frequent omission of critical metadata such as the coordinate reference system. This agent aims to resolve these issues by translating each free-form query into a normalised JSON schema that explicitly records the analytical task category, target feature classes, spatial predicates, grouping attributes, and quality constraints. This agent employs LLaMA-3 language model guided by a prompt template optimised for semantic-role labelling and ontology alignment. The structured output confers two principal benefits: it eliminates the need for downstream components to interpret unstructured prose, and it constitutes a machine-verifiable contract that can be checked for completeness and logical consistency. For the user query, this agent can output a json like this: `{‘intent’: ‘calculate total length of rivers within urban areas’, ‘action’: ‘spatial intersection’, ‘entities’: {‘primary’: ‘rivers’, ‘secondary’: ‘urban areas’}, ‘parameters’: {‘output_format’: ‘vector’, ‘geometry_type’: ‘LineString’, ‘aggregation’: ‘sum’}, ‘metric’: ‘total_length_meters’}`.
- (3) Data Digestion Agent: The Data Digestion Agent crawls the user-defined workspace using recursive file inspection and the QGIS API. For each recognised geospatial file, it extracts geometry type, CRS (via `QgsCoordinateReferenceSystem`), bounding box, and attribute schema through `QgsVectorLayer` or `QgsRasterLayer` introspection. The resulting metadata are serialised into a JSON list of candidate layers. If mandatory inputs referenced in the intent schema are missing or mismatched, the agent flags the issue early and halts execution, preventing invalid downstream calls. This agent ensures data readiness by browsing the user-defined workspace and retrieving each layer’s path, geometry type, CRS, bounding box, and attribute schema. Then, it identifies the suitable layers required for the current task. If no valid layers are found it flags this explicitly. This early validation prevents common downstream errors, shortens the debugging cycle, and allows the Reasoning Planner to make informed decisions based on data types, such as excluding point-specific methods for polygonal layers. An example of the output can be `{‘selected_layers’: ‘rivers’: {‘path’: ‘/workspace/data/rivers.shp’,`

'geometry_type': 'LineString', 'crs': 'EPSG:3006', 'feature_count': 482, 'attributes': ['id', 'name', 'length_m']], 'urban_areas': {...}}, 'metadata': {...}, 'explanation': '...}

- (4) Knowledge Retrieval Agent: The RAG agent implements a retrieval-augmented generation (RAG) mechanism to enrich model reasoning with domain-specific knowledge that cannot be inferred directly from the user query alone. The retrieval corpus consists of PyQGIS API documentation with QGIS algorithm manuals and representative usage examples, segmented at the level of individual tools or functions. Documents are indexed in a vector store using a hybrid retrieval strategy that combines sparse BM25 ranking with dense-vector similarity search, thus enabling robust retrieval across both keyword-driven and semantic queries. Dense embeddings are computed offline. In the proposed architecture, knowledge retrieval is tightly coupled with the planning process rather than invoked as an optional fallback. After intent interpretation and data digestion, the retrieval agent operates alongside the planning agent to provide contextual grounding for tool selection, parameter specification, and API usage as candidate workflow steps are being constructed. Retrieved passages are incrementally injected into the context of the planning and code-generation agents, allowing both the workflow structure and the generated code to be informed by executable tool knowledge while the plan is being formed. Only the top-k retrieved passages are included as contextual evidence and are not executed directly. The knowledge base used in this work is not intended as a standalone or comprehensive geospatial knowledge repository. Instead, it functions as a task-oriented, execution-grounding resource tailored specifically to support PyQGIS-based workflow planning and code generation. Its content is limited to operational knowledge required for selecting appropriate processing algorithms, configuring parameters, and using the PyQGIS API correctly. Accordingly, the knowledge base is maintained as a lightweight, document-level resource rather than as a centralised or publicly curated platform.

This design differs fundamentally from existing resources such as GEE-OPs (Hou et al. 2025d) or GeoFub (Hou et al. 2025e), which are primarily developed for the GEE ecosystem and aim to provide reusable repositories of Earth Engine operations or workflows for discovery and reuse. While such resources are valuable within cloud-based, declarative execution environments like GEE, they are not designed to support execution-grounded reasoning in desktop GIS systems or agent-based workflow construction. In contrast, our knowledge base is tightly coupled to the PyQGIS execution environment and the internal agent workflow, and is optimised for contextual grounding during autonomous planning and code synthesis rather than for cataloguing predefined operations.

- (5) Reasoning Planner Agent: It translates user intent into an executable geoprocessing workflow by integrating the parsed query, validated data layers, and retrieved knowledge. Through a structured chain-of-thought process, it produces three key artefacts: (i) a human-readable explanation of the reasoning steps, (ii) a JSON-encoded plan detailing algorithms, inputs, and parameters. This explicit representation allows deterministic code generation by the Executor and supports precise fault localisation by the Reflector through structural comparison of expected vs. actual outcomes. The generated plan can be similar to this:

```
{'step': 1, 'description': 'Access the 'rivers' and 'urban_areas' layers from the workspace and register them in the QGIS environment.', 'tool': 'PyQGIS API call (QgsVectorLayer)', 'inputs': ['rivers.shp', 'urban_areas.shp'], 'outputs': ['rivers_layer', 'urban_layer']}, {'step': 2, 'description': 'Perform a spatial intersection to extract river segments that fall within urban areas.', 'tool': 'native:intersection', 'inputs': ['rivers_layer', 'urban_layer'], 'outputs': ['rivers_in_urban']}, ...}
```
- (6) Code Generator and Executor Agent: It translates the planner's abstract workflow into PyQGIS code, identifies the necessary processing algorithms, and runs them in a headless QGIS environment. Both success artefacts (e.g. generated layers, figures) and failure artefacts (e.g. tracebacks, stdout/stderr, assertion violations) are captured and packaged into an execution result json object. This output underpins subsequent reflection, visualisation, and quality metrics. By isolating execution from reasoning, the agent simplifies error diagnosis and maintains clear boundaries between logic and runtime behaviour.
- (7) Reflector Agent: It enables self-healing by analysing the execution result json and the full history of agent outputs to assess whether the success criteria were met. These criteria are execution-aware and include successful completion of GIS operations, structural validity of intermediate and final outputs

(e.g. expected geometry types, attribute presence), and consistency between the executed workflow and the original task intent. On failure, it performs pattern-based diagnostics to trace the error to a specific upstream agent (e.g. mapping missing layers to the Data Ingestion Agent). This diagnosis relies on the logs from all the agents and their outputs, rather than attempting full semantic interpretation of the final geospatial files or results. It then generates a focused improvement prompt for corrective action and returns a structured json: a successful flag, the responsible agent's ID, and a new prompt asking the agent to fix the specific error. By targeting only the agent associated with the diagnosed failure, the reflector avoids unnecessary re-execution of unaffected components. This allows the orchestrator to trigger selective re-execution, avoiding redundant computation. An example of output generated by the agent can be: `{'ok': false, 'root_cause': 'Planner omitted CRS harmonisation before intersection; inputs use EPSG:4326 and EPSG:3006.', 'fault_source': 'Reasoning Planner Agent', 'improvement_prompt': 'Revise the plan to insert an explicit CRS check and a reprojection step prior to intersection. Reproject all inputs to EPSG:3006 before intersection. Include the new steps and updated outputs in the JSON plan.', 'reflection': 'Intersection returned empty features while logs show mixed CRS. The planner should ensure CRS alignment as part of the workflow. Adding a reprojection step before intersection will produce valid overlaps.'}`.

The agents collaborate through a message-passing protocol orchestrated by the LangChain/SmolAgent framework, which maintains contextual memory and supervises task delegation. Each agent produces structured outputs (in JSON format) that serve as explicit inputs to the next agent so to ensure modular interoperability and traceable reasoning. Collaboration is further reinforced through a feedback mechanism, where the Reflector agent analyses execution outcomes and dispatches targeted corrective prompts to the responsible upstream agent in order to enable iterative refinement and self-healing of the workflow.

To summarise, by integrating retrieval-augmented generation, multi-agent reasoning, and iterative code refinement, the proposed system ensures both correctness and adaptability in handling complex geospatial queries. The modular architecture allows for easy extension, enabling future enhancements such as reinforcement learning-based agent optimisation or integration with additional geospatial processing frameworks. This approach represents a significant advancement in AI-assisted geospatial analysis and aims to bridge the gap between natural language understanding and automated geospatial computation.

3.3. System implementation

We selected PyQGIS as the target platform primarily because it is (i) an open-source interface with comprehensive tool coverage, (ii) tightly integrated with the widely used QGIS platform, and (iii) representative of a wide-featured geoprocessing environment while remaining accessible for experimentation. We acknowledge that PyQGIS is not fully representative of all geospatial coding ecosystems. The implementation approach adopts a modular design that integrates LLMs with other language models, task-specific tools, and spatial analysis software to enhance performance and adaptability across diverse tasks. We also implement a multi-agent system, with each agent specialised in tasks such as query decomposition, data retrieval, or code generation. The full architecture is detailed in the next section. This framework supports advanced geospatial analysis by combining CoT reasoning, RAG, data understanding, automated code execution, and debugging. These components work together to address the complexities of geospatial workflows with improved accuracy and efficiency.

Agent Framework Implementation: The system was built with a focus on robust integration between language models and geospatial tools. The environment setup required configuration of dependencies including mainly LangChain, SmolAgents, Ollama, and QGIS. LangChain and SmolAgents were used to orchestrate agents, specifically REACT agents which follows this loop: reasoning → action → observation. QGIS, accessed via Python bindings, provides the geospatial processing capabilities. Each operation was implemented as a modular tool within the system. Regarding the internal structure, each agent includes in addition to the task-specific LLM core two components: a context memory module retains the current task state, key metadata, prior actions/results and retrieved knowledge, enabling persistent reasoning across steps. Third, a set of tool interfaces gives the agent structured access to domain-specific capabilities such as PyQGIS API bindings, vector store retrieval (RAG), and other geospatial utilities. Finally, the output

interface returns a JSON-formatted payload encapsulating the agent's decision, executed code or tool call, and resulting observation, thereby enabling downstream agents to parse and continue the workflow seamlessly.

Internally, the agent comprises: (i) a task-specific LLM core guided by structured prompts that iteratively reason about subtasks; (ii) a memory module that tracks state, previous actions, observations, and context; (iii) a defined set of tools (i.e. PyQGIS, RAG) that the agent can call as actions; and (iv) a structured output interface which can be either a JSON tool-call specification (in ToolCallingAgent variants) or a code snippet (in CodeAgent variants) which ensures downstream agents can parse and act on the result in a uniform way.

LLM Reasoner Fine-Tuning using CoT: Chain-of-Thought (CoT) reasoning is used to enhance the LLM's ability to solve complex geospatial queries by decomposing tasks into smaller, intermediate steps. This ensures the correctness of the GIS workflow and accuracy for operations such as proximity calculations, buffer analyses, and coordinate transformations. CoT ensures that each step contributes to the final result, making it central to this methodology. Unlike prior instruction-tuned baselines which perform direct text-to-code translation, our approach introduces a two-fold fine-tuning design. The first stage fine-tunes the DeepSeek R1 LLM on Query-CoT pairs to reason through CoT prompting to generate a structured plan from the user query, while the second stage fine-tunes the model to translate both the query and the generated plan into executable PyQGIS code. This dual optimisation jointly enhances reasoning coherence and code correctness, effectively bridging conceptual planning and implementation.

To effectively fine-tune DeepSeek-R1 for this kind of reasoning, we begin by tokenizing text-based fields (prompts, Chain-of-Thought reasoning) using a word tokenizer optimised for natural language processing, while encoding PyQGIS scripts with a code-specific tokenizer to preserve syntax structure and indentation. The latter tokenizer was initialised from the base model's vocabulary and extended with PyQGIS-specific keywords, function names, and common API tokens to ensure accurate encoding of code syntax and geoprocessing tool identifiers. Parameter-Efficient Fine-Tuning, specifically Quantised Low-Rank Adaptation (QLoRA) is used to fine tune the LLM. QLoRA introduces trainable low-rank matrices into the layers of a pre-trained model to effectively capture task-specific information while keeping the original model weights frozen. This approach is grounded in the observation that updates to the model weights during adaptation often reside in a low-dimensional subspace. By constraining the weight updates to be low-rank, LoRA significantly reduces the number of trainable parameters, thereby enhancing fine-tuning efficiency.

The model is optimised using the Cross-Entropy Loss function for text generation, ensuring alignment between prompts and correct GIS procedures, while Execution Accuracy is used as a core evaluation metric which measures whether generated scripts run successfully and produce the expected spatial outputs. Hyperparameters include a learning rate ($2e-5$) with linear warm-up, batch size 32, and gradient accumulation steps (4–8) for stable optimisation. Temperature tuning (0.2–0.8) is employed to balance deterministic output (low temperature) for GIS workflows and creative flexibility (higher temperature) when handling ambiguous prompts. Additional parameters such as top-k sampling ($k=40$) and top- p nucleus sampling ($p=0.9$) are applied to refine generation diversity while preventing excessive randomness. The final model undergoes validation using 20% of the data in order to ensure it generalises well across diverse QGIS operations. All experiments were conducted using 4 NVIDIA A100 GPUs (80 GB VRAM), Intel Xeon Gold 6338 (Ice Lake) processors (2×32 cores), and 256 GB RAM with Python 3.10 and QGIS 3.34 API.

Agent design: The agents were implemented following the ReAct paradigm, in which the LLM iteratively alternates between acting steps (calling predefined functions) and analysing the outcome of the functions. Each tool represents a parameterised function, (such as 'list files' and 'inspect file' functions in the case of Data Digestion agent), which allows the agent to perform concrete geospatial operations like listing available datasets or reading file metadata (CRS, geometry type, attributes, etc.). The reasoning-acting cycle is bounded by a maximum number of steps to prevent uncontrolled reflection loops or indefinite reasoning. In our experiments, the default value was kept which is 6 iterations.

The prompt design uses a structured, instruction-based format that explicitly defines (i) the user's natural language request, (ii) contextual outputs from preceding agents, and (iii) the precise task the model should accomplish, such as identifying relevant geospatial layers within a given directory. Each prompt enforces a strict JSON schema with predefined keys: `selected_layers`, `metadata`, and `explanation`. An example JSON block is embedded in the prompt to illustrate the correct output structure. This design

ensures that the model's response is deterministic, machine-readable, and easily parsed for downstream use.

Debugging Component: Automated debugging mechanisms ensure smooth execution of the generated code by identifying and resolving errors. Rather than relying on a traditional step-by-step debugging module, the system uses a lightweight post-execution reflection mechanism to analyse the outcome of the execution phase and identify the likely source of failure. After each run, the logs output is inspected by a dedicated agent to determine whether the task succeeded and, if not, which agent in the pipeline may have caused the failure. This design supports fault tolerance in a loosely coupled agent system, prioritising interpretability, low overhead, and incremental recovery without centralised debugging logic or hardcoded validation rules. It also allows error attribution in settings where failures may result from silent logic mismatches rather than obvious syntax or runtime exceptions.

Additional Components: To enhance the usability and functionality of the system, additional features have been incorporated. Visualisation tools, mainly Folium, are used to present results in an interactive and intuitive format. A user feedback loop collects insights to refine reasoning capabilities, improve knowledge retrieval, and optimise code generation.

3.4. Evaluation

To rigorously assess the performance of the proposed multi-agent geospatial analysis system, we employed a comprehensive evaluation framework designed to capture both code generation quality and autonomous agent behaviour across varying task complexities. Tasks were stratified into four complexity levels based on the number of geoprocessing tools involved (1, 2, 3, and 4–7), and each was evaluated over three runs to account for LLM variability. Since each agent must make independent decisions and the system relies on iterative self-correction, we considered both one-shot and multi-attempt evaluation settings, with up to five refinement iterations allowed per task. Our evaluation metrics included:

- Execution performance measured using Pass@1, Pass@5, Success@final (the percentage of tasks eventually completed), and the average number of iterations to success, to assess convergence efficiency.
- Self-healing ratio, which quantifies the proportion of tasks that failed on the first attempt but were successfully completed after iterative refinement.
- Semantic correctness and tool selection accuracy, determined through manual inspection of outputs to verify whether the generated results fulfilled the user's intent and whether appropriate geoprocessing tools were selected and correctly applied.
- Syntax similarity is assessed using AST-based Jaccard similarity between the generated and reference scripts to capture how closely the system reproduced structurally accurate code across complexity levels.
- Execution efficiency, recorded as the average runtime per task for each system variant, to evaluate trade-offs between performance and responsiveness.
- System variant comparison, which included the full multi-agent architecture, variants without fine-tuning and RAG, and a single-agent GPT-4 baseline, to isolate the contributions of key architectural components.
- Error attribution, comparing manual and Reflector Agent-based identification of error sources, to understand which components most frequently contributed to failure and how accurately they were diagnosed.

This diverse set of metrics ensures that we evaluate not only end-to-end correctness, but also the efficiency, robustness, syntax, and error recovery ability of the system, thus providing a holistic view of its practical performance in realistic GIS automation tasks.

4. Results

To assess the effect of task complexity on system performance, we stratified tasks by the number of geospatial tools required to complete them; 1, 2, 3, and 4–7 tools. Because the language model exhibits

stochastic behaviour, we repeated the evaluation three times and reported the averaged metrics: Pass@1, Pass@5, Success@final, and Average Iterations to Success as shown in Table 3.

4.1. Performance across task complexity levels

Table 3 shows that performance degrades as task complexity increases. For single-tool tasks, the system performed nearly flawlessly, achieving 87.5% Pass@1 and 100% final success, with an average of only 1.17 iterations needed to converge. However, for tasks requiring two tools, performance dropped noticeably, with Pass@1 falling to 70.83% and Success@final to 87.5%, indicating that even modestly more complex workflows introduce room for ambiguity in tool sequencing or parameter configuration. For three-tool workflows, Pass@1 drops further to 54.16%, and final success reaches 75%, with an average of 2.22 iterations to succeed. Tasks requiring four to seven tools represent the most challenging category. Only 25.64% of these were solved on the first attempt, and even with multiple retries (up to five), Success@final remained limited at 51.28%, requiring nearly three full iterations on average to reach a valid solution for the solvable tasks.

The self-healing ratio metric shows that the multi-agent system is capable of recovering from initial failures through iterative refinement, particularly for simpler tasks. For one- and two-tool workflows, the system successfully corrected 100% and 57% of initially failed tasks, respectively, demonstrating moderate to strong self-repair capabilities. However, for more complex tasks involving three or more tools, the self-healing ratio declines to 45% and 35%, which indicates that deeper reasoning and more robust debugging strategies are needed to effectively handle longer geospatial workflows. An example of a successful healing cycle is the query ‘Find restaurants within 500 m of hotel Cheraton.’ In the first run, the workflow failed with a KeyError caused by an attribute-name mismatch (*name* vs. *hotel_name*) in the hotels layer. The Reflector Agent correctly attributed the issue to the Data Digestion Agent, prompting it to re-scan the layer metadata and expose the actual attribute schema. After the agent updated the field reference and re-executed the workflow, the pipeline completed successfully, generating the correct selection of nearby restaurants within the specified distance. So, the chain was: Iter-1: EXEC FAIL, Error: KeyError: field ‘name’ not found, Reflector: fault_source = ‘Data Ingestion Agent’, Action: Re-scan layers; include full attribute schemas, Iter-2: Successful execution. Across all complexity levels, the framework achieved an overall execution success rate of 68% which is the weighted average of the Success@final metric in Table 2.

4.2. Semantic correctness and tool selection accuracy

In addition to evaluating execution success, we manually inspected the outputs to assess semantic correctness, i.e. whether the results were not only executed, but also correctly fulfilled the user’s intent and to evaluate tool usage accuracy, defined as whether the correct tools were selected and used with the appropriate parameters.

As shown in Table 4, semantic correctness degrades sharply with increasing task complexity. While all one-tool tasks produced correct results (100%), only 26.9% of tasks involving four or more tools were semantically valid. Overall, just 26 out of 50 tasks (52%) yielded correct outputs that correctly solving the

Table 3. Execution evaluation of the multi-agent approach.

Number of tools	@pass1	@pass5	Success@final	Average iterations to success	Self-healing ratio
1	87.5	100	100	1.17	1
2	70.83	83.33	87.5	1.48	0.57
3	54.16	66.66	75	2.22	0.45
4–7	25.64	43.58	51.28	2.85	0.34

Table 4. Semantic evaluation of the multi-agent approach.

Task complexity	Total tasks	AST jaccard similarity	Semantically correct (count)	Semantically correct (%)	Tool usage correct (count)	Tool usage correct (%)
1 tool	8	0.84	8	100%	8	100%
2 tools	8	0.76	7	87.5%	7	87.5%
3 tools	8	0.68	4	50%	6	75%
4–7 tools	26	0.35	7	26.9%	11	42.3%
Total	50	–	26	52%	32	64%

analysis, despite a higher functional success rate of 68%. This gap underscores the system’s tendency toward silent logical failures as workflows become more complex.

Analysing tool usage independently of output correctness, 32 out of 50 tasks (64%) used the correct tools in the correct sequence, despite parameter errors. In many multi-step tasks, semantic failures stemmed from selecting technically valid but semantically inappropriate tools. For example, when asked to identify buildings within 500 m of schools, the system used the native:clip algorithm instead of performing a buffer followed by select by location. While the resulting script executed successfully, it returned only the directly intersecting features rather than those within the specified distance. The sharpest decline occurred in 4–7 tool tasks, where only 42.3% used all tools correctly, highlighting challenges in long-chain reasoning and parameter tracking. To assess structural alignment, we computed AST-based Jaccard similarity across task complexities. Syntax similarity dropped from 0.84 in one-tool tasks to 0.35 in complex ones, which indicates the decline in semantic correctness and indicating greater structural divergence in longer workflows.

To evaluate how effectively the Reflector Agent identifies failure sources within the multi-agent system, we compared its diagnoses to manual inspection across 50 failed tasks. As shown in Table 5, the Data Ingestion Agent was identified by humans as the most common source of failure (46%), often due to incorrect or missing data layer. In contrast, the Reflector attributed the highest share of errors to the Code Generator Agent (46%) which reflects the fact that execution failures tend to surface at the final stage of the pipeline, even when the underlying cause originates earlier in the workflow. Notably, the Reflector correctly captured responsibility for 10–28% of failures in upstream agents, such as Planning and Query Parsing. In addition, the Reflector attributed 6% of failures to the RAG Agent, whereas no such failures were flagged during manual inspection. This discrepancy is due to the difficulty of subjectively assessing retrieval quality and that the impact of RAG outputs on downstream failures can be subtle or indirect. The Reflector’s identification of these cases suggests it may be capable of detecting latent issues in knowledge retrieval that are otherwise hard to trace through manual analysis alone.

4.3. Comparative evaluation of system variants

We evaluate four system configurations as part of an ablation analysis to isolate the effects of fine-tuning, retrieval grounding, and agent collaboration: Multi-Agent Full, Multi-Agent without fine-tuning, Multi-Agent without retrieval (RAG), Single-Agent GPT-4. In addition, we include a comparative evaluation against ChatGeoAI (Mansourian and Oucheikh 2024), which is selected as the closest prior system to our approach, as it focuses on translating natural language queries into executable PyQGIS code. ChatGeoAI relies on a single fine-tuned LLM, LLaMA 2 model, that directly translates a user query into PyQGIS code in a one-shot manner, without explicit task decomposition, external knowledge retrieval, or iterative refinement. In contrast, our framework adopts a multi-agent architecture in which specialised agents collaboratively handle intent parsing, data inspection, retrieval-augmented planning, code execution, and reflective self-correction. With respect to intent understanding, ChatGeoAI combines prompt-guided named entity recognition with geospatial ontology grounding to map user queries to GIS concepts, layers, and attributes, which can be advantageous for resolving semantic intent and attribute-level references. However, this semantic grounding is embedded within a single text-to-code generation step, leaving task decomposition implicit. In contrast, our framework explicitly decomposes queries into structured, multi-step workflows via a dedicated planning agent that is potentially enabling finer-grained control over tool sequencing, parameter propagation, and intermediate validation. The structural and functional

Table 5. Error attribution by manual inspection vs. reflector agent.

Agent	Manual inspection		Reflector identification	
	Error Count	Error Share (%)	Error Count	Error Share (%)
Data ingestion agent	23	46	14	28
Code generator agent	17	34	23	46
Planning agent	7	14	5	10
Query parser agent	3	6	5	10
RAG agent	–	–	3	6

Table 6. Structural and functional comparison between ChatGeoAI and the proposed framework.

Aspect	ChatGeoAI	Proposed framework
Architecture	Single-agent pipeline	Multi-agent system
Intent understanding	NER with geospatial ontology grounding	Dedicated intent interpreter agent
Task decomposition	Implicit within code generation	Explicit planning stage via reasoning planner agent
Workflow structure	Linear PyQGIS script	Structured workflow plan (JSON) with explicit tool dependencies
Knowledge grounding	Prompt-based	RAG over GIS documentation (PyQGIS docs)
Execution model	One-shot execution	Iterative execution via automated reflection and error handling
Intermediate validation	Not supported	Supported

Table 7. Comparison of multi-agent variants and GPT-4.

System variant	Metric	1 Tool	2 Tools	3 Tools	4–7 Tools
Multi-Agent full	Pass@5 (%)	100	83.3	66.6	43.6
	Sem. Correct (%)	100	87.5	50.0	26.9
Multi-Agent (no fine-tuning)	Pass@5 (%)	83.3	70.8	50.0	28.2
	Sem. Correct (%)	83.3	62.5	37.5	19.2
Multi-Agent (No RAG)	Pass@5 (%)	91.6	75.0	58.3	33.3
	Sem. Correct (%)	91.6	75.0	54.1	23.0
Single-agent GPT-4	Pass@5 (%)	87.5	66.6	37.5	22.4
	Sem. Correct (%)	87.5	58.3	33.3	15.3
ChatGeoAI	Pass@5 (%)	70.8	50.0	28.2	14.0
	Sem. Correct (%)	62.5	50.0	23.0	13.3

comparison of our approach with ChatGeoAI is summarised in Table 6. The quantitative comparison is performed afterwards based on both Pass@5 success rate and semantic correctness, stratified by task complexity.

Table 7 shows the comparison results. The Multi-Agent Full system consistently achieves the highest performance across all task classes. It reaches 100% success and correctness for 1-tool tasks and maintains relatively strong results for 2- and 3-tool tasks, with Pass@5 of 83.3% and 66.6%, and semantic correctness of 87.5% and 50%, respectively. However, its performance drops significantly for the most complex category (4–7 tools). This highlights the challenge of long, multi-step workflows. Removing fine-tuning reduces both success and correctness across all task levels, particularly in 3-tool and 4+ tool tasks, indicating that task-specific adaptation is crucial for coherent decomposition and tool chaining. In this setting, failures are more frequently caused by incorrect parameterisation, API misuse, or invalid field references, even when the overall workflow structure is reasonable. Similarly, removing RAG degrades performance, but less drastically, showing that retrieval helps primarily in parameter grounding and knowledge-intensive tasks. The Single-Agent GPT-4 baseline performs reasonably on simple tasks but degrades sharply with task complexity. Its Pass@5 drops to 22.4% and semantic correctness to 15.3% for 4–7 tool workflows. Similarly, when evaluated on our benchmark, ChatGeoAI attained a Pass@5 of 70.8% for single-tool tasks but declined sharply to 14.0% for workflows involving 4–7 tools, with corresponding semantic correctness falling from 62.5% to 13.3%. These failures are predominantly associated with operation sequencing, data identification, attribute referencing, and parameter usage.

These results confirm that without structured decomposition and retrieval, single agents struggle to maintain coherence and correctness in multi-step geospatial analyses. Overall, the results demonstrate that both fine-tuning and RAG enhance multi-agent performance, with fine-tuning providing the greater benefit, particularly for complex, multi-step tasks. They confirm also that task decomposition and tool planning become increasingly critical as complexity grows. In addition, they indicate that multi-agent collaboration alone does not constitute an optimal solution, and suggest that fine-tuning and agent collaboration address complementary failure modes: fine-tuning primarily enhances local correctness, while multi-agent orchestration strengthens system-level robustness, recoverability, and autonomy.

To assess the efficiency and responsiveness of each system configuration, we evaluated the average execution time per task and the average number of iterations to success, which are critical for understanding the practical feasibility of deploying these systems in real-world, time-sensitive geospatial applications. Table 8 presents the average execution time per task and the average number of iterations to success for each system configuration. The multi-agent Full system achieves a balance between performance and efficiency, requiring only 1.9 iterations on average with an execution time of 135.98 seconds per task. Removing fine-tuning results in longer convergence (2.1 iterations) and higher time cost (150.30 seconds), while removing retrieval (RAG) leads

Table 8. Execution efficiency of system variants.

System variant	Avg. time per task (s)	Avg. iterations to success
Multi-Agent Full	135.98	1.9
Multi-Agent (no fine-tuning)	150.30	2.1
Multi-Agent (no RAG)	129.62	2.3
Single-Agent GPT-4	64.40	2.8

to a greater number of retries (2.3) but slightly lower overall time (129.62 seconds), suggesting that failed executions were sometimes faster due to earlier terminations. The Single-Agent GPT-4 system exhibits the lowest execution time (64.40 seconds), but also the highest number of iterations (2.8), which reflects more frequent failed attempts and lack of intermediate debugging. This highlights a trade-off between speed and reliability across systems.

5. Discussion

The results of this study offer important insights into the capabilities and limitations of using a multi-agent LLM framework for autonomous geospatial analysis. This discussion critically examines the findings in light of the system's design objectives and evaluation metrics, with particular attention to how the architecture handles task complexity, agent specialisation, reasoning quality, tool selection, error recovery, and real-world applicability.

5.1. Performance across task complexity levels

The performance evaluation indicates that while the multi-agent system is effective for atomic or shallow workflows, it becomes increasingly error-prone with longer, multi-step pipelines. For one- and two-tool workflows, the multi-agent framework achieved high Pass@1 and Pass@5 rates alongside near-perfect semantic correctness and demonstrate effectiveness in translating straightforward natural language queries into valid geospatial procedures. However, as task complexity increases, particularly in cases involving three or more tools and chained spatial operations, performance drops significantly. Notably, the prominence of Data Ingestion and Planning Agent errors (60% combined in manual inspection, Table 5) is most pronounced in tasks involving longer tool chains, indicating that failures increasingly arise from early-stage decisions whose effects propagate across multiple processing steps. This decline suggests that even with specialised agents and CoT reasoning, the architecture struggles with long-range planning due to cumulative reasoning errors, parameter misalignment, and difficulties in maintaining context across steps. The low performance manifests as incorrect tool ordering, missing parameters, inappropriate use of functions, or misuse of intermediate outputs. For example, in several 4–7 tool workflows, an early planning error, such as applying a spatial filter that unintentionally excludes required features or alters geometry structure, produced intermediate layers with incompatible attributes or geometries. Because subsequent operations remained syntactically valid with respect to these intermediate outputs, the remaining tools executed without runtime errors, but the final results were semantically incorrect. In some cases, iterative reflection successfully corrected the immediate cause of failure (e.g. by adjusting tool parameters or replacing an operation), but the modification introduced secondary inconsistencies elsewhere in the workflow, such as mismatched inputs for downstream steps. This illustrates how, in long and tightly coupled tool chains, correcting one error can inadvertently expose or create another, which makes complete recovery increasingly difficult as workflow complexity grows.

Such findings align with known limitations of LLMs in multi-step reasoning tasks, especially in domains requiring precise syntactic and semantic alignment, such as geospatial analysis. The system maintained high accuracy for simple workflows, but performance degraded with increasing task complexity, primarily due to reasoning drift and parameter misalignment. This is generally aligned with a study using copilot for remote sensing workflows (Lee et al. 2025) where the multi-agent system achieved up to a 17% improvement in agentic correctness compared to single-agent baselines as task complexity increased.

5.2. Tool selection and semantic reliability

While execution success was relatively high, only 52% of tasks were semantically correct, despite a 68% execution success rate. This highlights the need to evaluate not just whether code runs, but whether it achieves the intended analysis. Notably, 64% of tasks used correct tools in the right order, yet semantic failures often resulted from selecting technically valid but contextually incorrect tools, especially in complex, multi-step (4–7) tasks involving multiple layers, projections, or spatial-attribute reasoning. These errors underscore the challenge of semantic precision in geospatial automation, where minor misalignments can yield misleading outputs. Improving this requires better tool-context alignment, stronger spatial logic grounding, and geospatial ontology integration. Enhancements to the Reasoning Planner and Code Generator with focus on task decomposition and parameter grounding, could significantly improve semantic reliability in workflows with over four steps. Additionally, validating selected tools or incorporating post-hoc plan checking may help catch semantic errors before execution.

5.3. Role of fine-tuning, RAG and reflector agent

The comparison of different system variants showed that both fine-tuning and RAG contribute meaningfully to performance. The absence of fine-tuning significantly reduced both success rates and semantic correctness, especially for complex tasks. This confirms the importance of domain-specific adaptation, particularly in helping the reasoning agent generate accurate decomposition steps aligned with geospatial logic. Similarly, removing the RAG component caused a moderate but consistent performance drop, especially in tasks requiring detailed parameter knowledge or tool documentation. Together, these results support the hypothesis that combining CoT reasoning with retrieval and fine-tuning creates a synergistic effect that improves both correctness and interpretability.

The inclusion of a Reflector Agent for post-hoc error detection and prompt revision significantly improved the system's robustness. The observed increase in success from Pass@1 to Success@final demonstrates the framework's ability to recover from initial errors through multiple iterations. However, the average number of iterations needed to achieve success also increased with task complexity, and in some cases, the system failed to recover at all. This suggests that while the self-healing mechanism is effective, it is not yet sufficient for reliably handling highly complex workflows. Future work could explore more advanced debugging strategies, such as backtracking workflows, learning from failure patterns, or using rule-based validation between iterations.

5.4. Cost vs efficiency analysis

Regarding the computation time efficiency, the results underscore an important trade-off in designing autonomous geospatial systems: while single-agent GPT-4 is considerably faster on average, it also required the most retries to succeed and demonstrated the lowest semantic correctness in prior evaluations. Its low latency may be suitable for simple, low-risk tasks, but the lack of reasoning and validation stages may lead to greater unreliability in complex workflows. The multi-agent Full system is slower, and the higher runtime is largely due to orchestration overhead and sequential agent operations. Notably, removing RAG reduced execution time slightly but increased retries which indicates that retrieval helps guide the agent toward more accurate actions earlier in the process. In contrast, removing fine-tuning increased both runtime and retries, which confirms that model adaptation plays a key role in guiding agents to produce more precise and executable steps on the first or second try.

5.5. Limitations and future directions

Several limitations point to areas for improvement. While the dataset of 500 tasks is a valuable benchmark, its diversity and coverage could be expanded to include more edge cases, rare tools, or 3D and temporal GIS tasks. In addition, chain-of-thought reasoning is sometimes shallow, missing implicit dependencies and spatial constraints. Execution could also benefit from better agent parallelism and caching. To handle complex, multi-step workflows more reliably, future work should focus on enhancing semantic reliability,

coordination, and adaptability. This includes improving long-chain decomposition via rule-based validation or task graphs, adding intermediate validation layers to prevent agent misalignment, and integrating spatial ontologies for better semantic grounding. Another limitation of the current framework is that the reflection mechanism focuses on execution-level signals and structural consistency checks rather than full semantic validation of geospatial outputs. While this design enables autonomous self-correction during runtime, it does not guarantee that all semantically incorrect results can be detected automatically, particularly in long and heterogeneous workflows. Addressing richer forms of semantic evaluation across diverse GIS output modalities remains an open challenge and an important direction for future work.

Future research can build on this work by integrating spatial ontologies and tool schemas to strengthen semantic reasoning and enable the system to better understand and align with geospatial concepts and operations. Reinforcement learning could also be used to train agents not only for task execution but for adaptive coordination and dynamic role assignment. This may make the system more responsive to ambiguous or multi-step queries. The use of multimodal LLMs could further enhance capabilities by allowing direct interpretation of maps and spatial context. In parallel, developing a self-improvement architecture where agents learn from past errors through experience replay could reduce redundancy and improve long-term reliability. Although this study centres on PyQGIS due to its open-source accessibility and transparent API structure, the instruction-tuning principles are model-agnostic. By abstracting tool operations as function-call templates, the same instruction-tuning strategy can generalise to ARCPY or GEE interfaces with minimal adaptation. So, to broaden applicability, future systems should support cross-platform code translation, allowing users to generate workflows for other platforms. Another aspect that can be examined in the future is scalability, including performance profiling, resource utilisation, and integration with distributed or cloud-based execution environments. Specifically, adaptive retrieval strategies and more detailed performance profiling to optimise the efficiency of the retrieval-augmented grounding mechanism can be examined to balance computational cost with robustness in autonomous GIS workflows.

In summary, this work provides a solid foundation for LLM-powered autonomous GIS systems. While limitations remain in semantic reasoning, long-chain execution, and tool alignment, the modular, multi-agent design shows measurable improvements over baseline approaches. With continued refinement, such systems have the potential to play a key role in the emerging landscape of intelligent Digital Earth infrastructures.

6. Conclusion

This study presents a novel multi-agent framework that integrates large language models with GIS tools to enable autonomous, natural language-driven geospatial analysis. Combining Chain-of-Thought (CoT) reasoning, Retrieval-Augmented Generation (RAG), and task-specific fine-tuning, the system translates high-level queries into structured PyQGIS workflows via specialised agents. It emulates human-like spatial reasoning, tool orchestration, and iterative debugging, advancing toward intelligent geospatial assistants. Experimental results show improved execution accuracy, tool selection, and semantic reliability over baseline models and simplified variants. The framework performs well on low- to moderate-complexity tasks, with self-healing mechanisms offering resilience to failure. However, performance declines with increased task complexity, highlighting challenges in long-chain reasoning, parameter propagation, and workflow alignment. Key contributions of this work are both methodological and practical and include a modular GIS automation architecture, an annotated benchmark dataset, and empirical validation of fine-tuning and external knowledge use. This work supports the Digital Earth vision by making spatial analysis more accessible and intelligent. Future directions include improving semantic robustness, scaling the system, and broadening support for diverse geospatial tasks and platforms, bringing us closer to AI-augmented tools for intuitive, user-friendly spatial data interaction.

Author contributions

CRediT: **Ali Mansourian:** Conceptualization, Investigation, Methodology, Project administration, Resources, Supervision, Visualization, Writing – review & editing; **Rachid Oucheikh:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This research received no funding.

ORCID

Ali Mansourian  0000-0001-6812-4307

Rachid Ouchekh  0000-0001-9996-9759

Data availability statement

The datasets collected during the current study are available from the corresponding author on reasonable request.

References

- Akinboyewa, T., Z. Li, H. Ning, and M. N. Lessani. 2025. "GIS Copilot: Towards an Autonomous GIS Agent for Spatial Analysis." *International Journal of Digital Earth* 18 (1): 2497489. <https://doi.org/10.1080/17538947.2025.2497489>.
- Bhandari, P., A. Anastasopoulos, and D. Pfoser. 2023. "Are Large Language Models Geospatially Knowledgeable? In *Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems*. p 1–4. <https://doi.org/10.1145/3589132.3625625>
- Christakis, N., and D. Drikakis. 2025. "Evaluating Large Language Models in Code Generation: INFINITE Methodology for Defining the Inference Index." *Applied Sciences* 15 (7): 3784. <https://doi.org/10.3390/app15073784>.
- Hadi, M. U., Q. A. Tashi, R. Qureshi, A. Shah, A. Muneer, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, and S. Mirjalili. 2023. "Large Language Models: A Comprehensive Survey of its Applications, Challenges, Limitations, and Future Prospects." *TechRxiv*, <https://doi.org/10.36227/techrxiv.23589741.v3>.
- Hou, S., J. Liang, A. Zhao, and H. Wu. 2025e. "GEE-OPs: An Operator Knowledge Base for Geospatial Code Generation on the Google Earth Engine Platform Powered By Large Language Models." *Geo-Spatial Information Science*, 1–22. <https://doi.org/10.1080/10095020.2025.2505556>.
- Hou, S., H. Jiao, Z. Shen, J. Liang, A. Zhao, X. Zhang, J. Wang, and H. Wu. 2025a. "Chain-Of-Programming (CoP): Empowering Large Language Models for Geospatial Code Generation Task." *International Journal of Digital Earth* 18 (1): 2509812. <https://doi.org/10.1080/17538947.2025.2509812>.
- Hou, S., H. Jiao, J. Liang, Z. Shen, A. Zhao, and H. Wu. 2025b. "GeoCogent: An LLM-based Agent for Geospatial Code Generation." *International Journal of Geographical Information Science*, 1–34. <https://doi.org/10.1080/13658816.2025.2549460>.
- Hou, S., Z. Shen, A. Zhao, J. Liang, Z. Gui, X. Guan, R. Li, and H. Wu. 2025c. "GeoCode-GPT: A Large Language Model for Geospatial Code Generation." *International Journal of Applied Earth Observation and Geoinformation* 138: 104456. <https://doi.org/10.1016/j.jag.2025.104456>.
- Hou, S., A. Zhao, J. Liang, Z. Shen, and H. Wu. 2025d. "Geo-FuB: A Method for Constructing an Operator-Function Knowledge Base for Geospatial Code Generation with Large Language Models." *Knowledge-Based Systems* 319: 113624. <https://doi.org/10.1016/j.knosys.2025.113624>.
- Huang, C., S. Chen, Z. Li, J. Qu, Y. Xiao, J. Liu, and Z. Chen. 2024. "GeoAgent: To Empower LLMs Using Geospatial Tools for Address Standardization. In *Findings of the Association for Computational Linguistics: ACL 2024*. L.-W. Ku, A. Martins and V. Srikumar, eds. p 6048–6063. <https://doi.org/10.18653/v1/2024.findings-acl.362>
- Hu, Y., G. Mai, C. Cundy, K. Choi, N. Lao, W. Liu, G. Lakhanpal, R.Z. Zhou, and K. Joseph. 2023. "Geo-knowledge-guided GPT models improve the extraction of location descriptions from disaster-related social media messages." *International Journal of Geographical Information Science* 37 (11): 2289–2318. <https://doi.org/10.1080/13658816.2023.2266495>.
- Jiang, Y., and C. Yang. 2024. "Is ChatGPT a Good Geospatial Data Analyst? Exploring the Integration of Natural Language into Structured Query Language Within a Spatial Database." *ISPRS International Journal of Geo-Information* 13 (1): 26. <https://doi.org/10.3390/ijgi13010026>.
- Lee, C., V. Paramanayakam, A. Karatzas, Y. Jian, M. Fore, H. Liao, F. Yu, R. Li, I. Anagnostopoulos, and D. Stamoulis. 2025. "Multi-Agent Geospatial Copilots for Remote Sensing Workflows." *arXiv preprint*. arXiv:2501.16254. <https://arxiv.org/abs/2501.16254>.
- Li, Z., and H. Ning. 2023. "Autonomous GIS: The Next-Generation AI-Powered Gis." *International Journal of Digital Earth* 16 (2): 4668–4686. <https://doi.org/10.1080/17538947.2023.2278895>.

- Lin, Q., R. Hu, H. Li, S. Wu, Y. Li, K. Fang, H. Feng, Z. Du, and L. Xu. 2024. "ShapefileGPT: A Multi-Agent Large Language Model Framework for Automated Shapefile Processing." *arXiv preprint*. arXiv:2410.12376. <https://arxiv.org/abs/2410.12376>.
- Liu, J., C. S. Xia, Y. Wang, and L. Zhang. 2023. "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation." *arXiv preprint*. <https://doi.org/10.48550/arXiv.2305.01210>.
- Mansourian, A., and R. Oucheikh. 2024. "ChatGeoAI: Enabling Geospatial Analysis for Public through Natural Language, with Large Language Models." *ISPRS International Journal of Geo-Information* 13 (10): 348. <https://doi.org/10.3390/ijgi13100348>.
- Manvi, R., S. Khanna, G. Mai, M. Burke, D. Lobell, and S. Ermon. 2024. "GeoLLM: Extracting Geospatial Knowledge from Large Language Models." *arXiv preprint*. arXiv:2310.06213. <https://arxiv.org/abs/2310.06213>.
- Mai, G., Huang, W., Sun, J., Song, S., Mishra, D., Liu, N., and Gao, S., et al. 2023. On the opportunities and challenges of foundation models for geospatial artificial intelligence. <https://doi.org/10.48550/arXiv.2304.06798>
- Ning, H., Z. Li, T. Akinboyewa, and M. N. Lessani. 2025. "An Autonomous GIS Agent Framework for Geospatial Data Retrieval." *International Journal of Digital Earth* 18 (1). <https://doi.org/10.1080/17538947.2025.2458688>.
- Raiaan, M. A. K., M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam. 2024. "A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges." *IEEE Access* 12: 26839–26874. <https://doi.org/10.1109/access.2024.3365742>.
- Singh, S., M. Fore, and D. Stamoulis. 2024. GeoLLM-Engine: A Realistic Environment for Building Geospatial Copilots. In *Proceedings of the 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. p 585–594. <https://doi.org/10.1109/cvprw63382.2024.00063>
- Stamoulis, D. and D. Marculescu. 2025. Geo-OLM: Enabling Sustainable Earth Observation Studies with Cost-Efficient Open Language Models and State-Driven Workflows. In *Proceedings of the ACM SIGCAS/SIGCHI Conference on Computing and Sustainable Societies (COMPASS '25)*. p 608–619. <https://doi.org/10.1145/3715335.3735494>
- Tucker, S. 2024. "A Systematic Review of Geospatial Location Embedding Approaches in Large Language Models: A Path to Spatial AI Systems." *arXiv preprint*. arXiv:2401.10279. <https://arxiv.org/abs/2401.10279>.
- Wang, S., T. Hu, H. Xiao, Y. Li, C. Zhang, H. Ning, R. Zhu, Z. Li, and X. Ye. 2024. "GPT, Large Language Models (LLMs) and Generative Artificial Intelligence (GAI) Models in Geospatial Science: A Systematic Review." *International Journal of Digital Earth* 17 (1): 2353122. <https://doi.org/10.1080/17538947.2024.2353122>.
- Wei, C., Y. Zhang, X. Zhao, Z. Zeng, Z. Wang, J. Lin, Q. Guan, and W. Yu. 2024. "GeoTool-GPT: A Trainable Method for Facilitating Large Language Models to Master GIS Tools." *International Journal of Geographical Information Science* 39 (4): 707–731. <https://doi.org/10.1080/13658816.2024.2438937>.
- Wei, J., X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. 2022. Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS 2022)*, Article 1800. p 1–14, Red Hook, NY: Curran Associates Inc.
- Wu, H., H. Jiao, S. Hou, J. Liang, Z. Shen, A. Zhao, Y. Qing, F. Jin, X. Guan, and Z. Gui. 2025. "GeoColab: An LLM-based Multi-Agent Collaborative Framework for Geospatial Code Generation." *International Journal of Digital Earth* 18 (2): 2569405. <https://doi.org/10.1080/17538947.2025.2569405>.
- Xu, W., C. Huang, S. Gao, and S. Shang. 2025a. "LLM-Based Agents for Tool Learning: A Survey." *Data Science and Engineering* 10: 533–563. <https://doi.org/10.1007/s41019-025-00296-9>.
- Xu, L., S. Zhao, Q. Lin, L. Chen, Q. Luo, S. Wu, X. Ye, H. Feng, and Z. Du. 2025b. "Evaluating Large Language Models on Geospatial Tasks: A Multiple Geospatial Task Benchmarking Study." *International Journal of Digital Earth* 18 (1): 2480268. <https://doi.org/10.1080/17538947.2025.2480268>.
- Yao, S., J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." *arXiv preprint*. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>.
- Zhang, Y., Z. He, J. Li, J. Lin, Q. Guan, and W. Yu. 2024a. "MapGPT: An Autonomous Framework for Mapping By Integrating Large Language Model and Cartographic Tools." *Cartography and Geographic Information Science* 51 (6): 717–743. <https://doi.org/10.1080/15230406.2024.2404868>.
- Zhang, Y., C. Wei, Z. He, and W. Yu. 2024b. "GeoGPT: An assistant for understanding and processing geospatial tasks." *International Journal of Applied Earth Observation and Geoinformation* 131: 103976. [10.1016/j.jag.2024.103976](https://doi.org/10.1016/j.jag.2024.103976).
- Zhang, J., C. Clairmont, X. Que, W. Li, W. Chen, C. Li, and X. Ma. 2025. "Streamlining Geoscience Data Analysis with an LLM-Driven Workflow." *Applied Computing and Geosciences* 25: 100218. <https://doi.org/10.1016/j.acags.2024.100218>.